

Python 科学计算教程

Contents

- 4.1. 为什么使用Python
- 4.2. Python 语言基础
- 4.3. 科学 Python 技术栈
- 4.4. 建议

欢迎学习这篇 Python 科学计算入门教程。本 notebook 将引导您了解 Python 编程语言的基本概念，然后介绍构成科学 Python 生态系统基础的核心库：NumPy、Pandas、Matplotlib 和 SciPy。

4.1. 为什么使用Python

传统而言，我们会采用MathCad或者Matlab等商业软件进行科学计算和数据分析。然而，Python作为一种开源编程语言，提供了许多优势：

传统而言，我们会采用MathCad或者Matlab等商业软件进行科学计算和数据分析。然而，Python作为一种开源编程语言，提供了许多优势，使其在学术界和工业界都备受欢迎。

4.1.1. 成本与开源优势

最显著的优势在于成本。Python及其所有科学计算库（如NumPy, SciPy, Pandas等）都是完全免费和开源的。这与Matlab等商业软件昂贵的许可证费用形成鲜明对比。开源特性不仅降低了个人、教育机构和企业入门的门槛，还意味着用户可以自由地查看、修改和分发代码，从而促进了技术的透明度和创新。

4.1.2. 强大的生态系统与通用性

Python拥有一个庞大且不断增长的生态系统。除了本教程将介绍的科学计算库，还有适用于机器学习（如 Scikit-learn, TensorFlow, PyTorch）、Web开发（如 Django, Flask）、自动化任务等各个领域的成熟库。这种通用性使Python成为一种强大的“胶水语言”，能够将不同部分无缝集

成在一起，例如，您可以先用Python进行数据分析和建模，然后直接用它构建一个Web应用来展示结果，而无需切换开发环境或语言。

4.1.3. 社区支持与可读性

Python拥有全球最大、最活跃的开发社区之一。这意味着当您遇到问题时，可以轻松地在Stack Overflow、GitHub或各种论坛上找到答案和支持。此外，Python以其简洁、清晰、易于阅读的语法而闻名，这使得代码更易于维护和协作，也大大降低了初学者的学习曲线。

4.1.4. 可复现性与自动化

在科学研究和工程计算中，结果的可复现性至关重要。Jupyter Notebook（就像您现在正在使用的这个环境）允许我们将代码、说明文本、方程式和可视化结果整合在一个文档中，极大地提高了工作的透明度和可复现性。同时，Python脚本可以轻松实现重复性任务的自动化，从而将工程师和研究人员从繁琐的工作中解放出来，专注于更具创造性的部分。

4.2. Python 语言基础

4.2.1. 变量和数据类型

```
# 数字（整数和浮点数）
x = 10          # 整数
y = 3.14        # 浮点数
z = x * y

print(f"x 的类型是 {type(x)}")
print(f"y 的类型是 {type(y)}")
print(f"z 的值是 {z}")
```

```
x 的类型是 <class 'int'>
y 的类型是 <class 'float'>
z 的值是 31.400000000000002
```

```
# 字符串
message = "你好，世界!"
name = 'Python'

greeting = message + " 我在学习 " + name + "。"
print(greeting)
```

你好，世界！我在学习 Python。

```
# 布尔值
is_learning = True
is_expert = False

print(f"我在学习吗? {is_learning}")
```

我在学习吗? True

4.2.2. 基本数据结构

```
# 列表：有序、可变（changeable）的项目集合。
my_list = [1, 2, 3, "apple", "banana"]

# 访问元素
print(f"第一个元素: {my_list[0]}")
print(f"最后一个元素: {my_list[-1]}")

# 修改列表
my_list.append("orange")
my_list[0] = "one"
print(f"修改后的列表: {my_list}")
```

第一个元素: 1

最后一个元素: banana

修改后的列表: ['one', 2, 3, 'apple', 'banana', 'orange']

```
# 字典：键值对的无序集合。
my_dict = {
    "name": "John Doe",
    "age": 30,
    "city": "New York"
}

# 通过键访问值
print(f"姓名: {my_dict['name']}")

# 添加新的键值对
my_dict["occupation"] = "Engineer"
print(f"更新后的字典: {my_dict}")
```

姓名: John Doe

更新后的字典: {'name': 'John Doe', 'age': 30, 'city': 'New York', 'occupation': 'Engi

4.2.3. 控制流

```
# If-elif-else 语句
age = 20

if age < 18:
    print("你是未成年人。")
elif age >= 18 and age < 65:
    print("你是成年人。")
else:
    print("你是老年人。")
```

你是成年人。

```
# For 循环：遍历一个序列（例如列表）。
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(f"我喜欢 {fruit}。")

# 你也可以使用 range() 循环指定的次数
for i in range(5):
    print(f"循环次数 {i}")
```

我喜欢 apple。
我喜欢 banana。
我喜欢 cherry。
循环次数 0
循环次数 1
循环次数 2
循环次数 3
循环次数 4

4.2.4. 函数

```
def greet(name):
    """这个函数向传入的参数 name 打招呼。"""
    return f"你好, {name}!"

# 调用函数
message = greet("Alice")
print(message)
```

你好, Alice!

```
def calculate_area(length, width):  
    """计算矩形的面积。"""  
    area = length * width  
    return area  
  
rect_area = calculate_area(10, 5)  
print(f"矩形的面积是 {rect_area}。")
```

矩形的面积是 50。

4.2.5. 导入库

4.3. 科学 Python 技术栈

4.3.1. NumPy: 数值 Python

```
# 按照惯例，NumPy 被导入并简写为 'np'  
import numpy as np  
  
# 从 Python 列表创建 NumPy 数组  
my_list = [1, 2, 3, 4, 5]  
np_array = np.array(my_list)  
  
print(f"这是一个 NumPy 数组: {np_array}")  
print(f"它的类型是: {type(np_array)}")  
  
# NumPy 数组允许逐元素操作  
doubled_array = np_array * 2  
print(f"数组元素翻倍: {doubled_array}")  
  
# 创建一个数字范围的数组  
range_array = np.arange(0, 10, 2) # 开始, 结束 (不包含), 步长  
print(f"从范围创建的数组: {range_array}")  
  
# 创建一个多维数组 (矩阵)  
matrix = np.array([[1, 2, 3], [4, 5, 6]])  
print("\n一个 2x3 矩阵:")  
print(matrix)  
print(f"矩阵的形状: {matrix.shape}")
```

这是一个 NumPy 数组: [1 2 3 4 5]
它的类型是: <class 'numpy.ndarray'>
数组元素翻倍: [2 4 6 8 10]
从范围创建的数组: [0 2 4 6 8]
\n一个 2x3 矩阵:
[[1 2 3]
 [4 5 6]]
矩阵的形状: (2, 3)

4.3.2. Pandas: 数据分析库

Pandas 提供了高性能、易于使用的数据结构和数据分析工具。Pandas 中的主要数据结构是 `Series` (一维) 和 `DataFrame` (二维)。DataFrame 就像一个电子表格或 SQL 表, 是 Pandas 中最常用的对象。

```
# 按照惯例, pandas 被导入并简写为 'pd'
import pandas as pd

# 从字典创建 DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}

df = pd.DataFrame(data)

# 显示 DataFrame
print("一个 pandas DataFrame:")
print(df)

# 获取数据摘要
print("\nDataFrame 信息:")
df.info()

# 选择单列 (这是一个 pandas Series)
print("\n选择 'Name' 列:")
print(df['Name'])

# 根据条件筛选数据
print("\n年龄大于 30 的人:")
print(df[df['Age'] > 30])
```

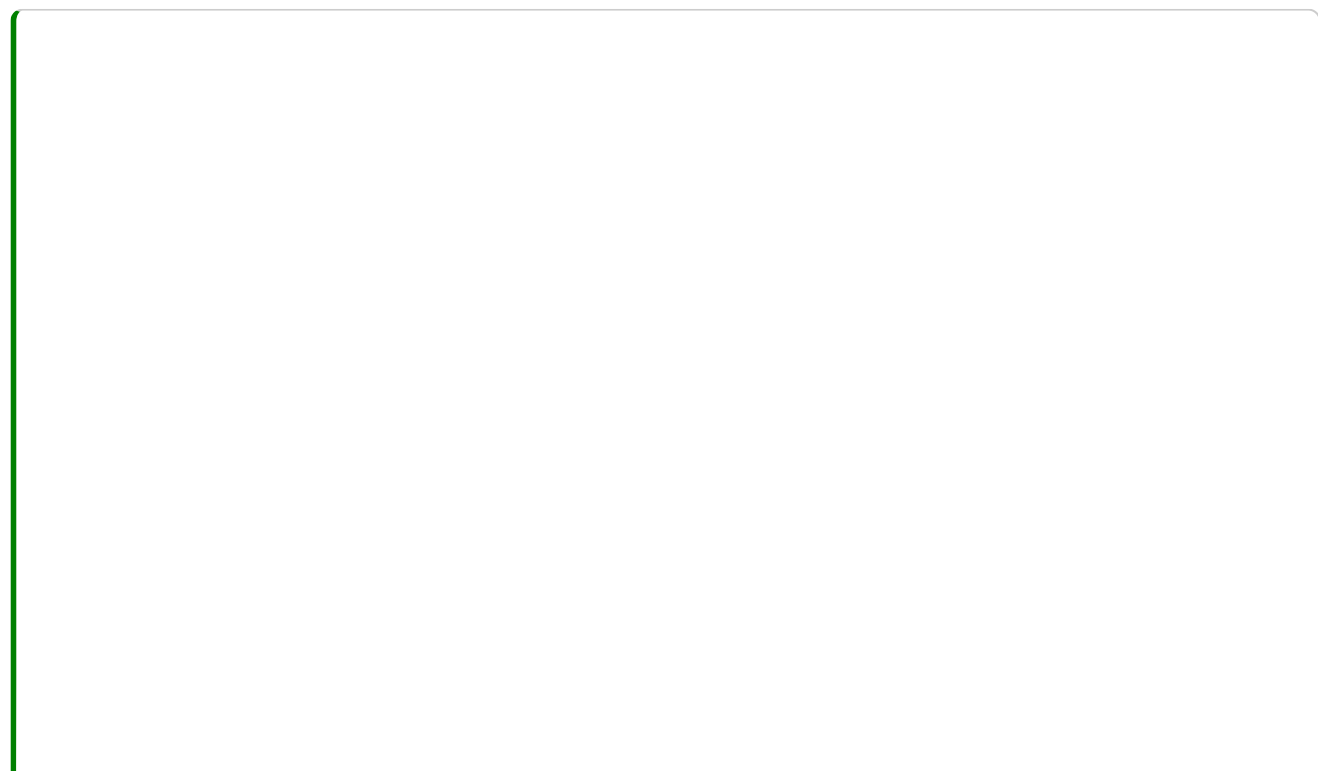
```

一个 pandas DataFrame:
   Name  Age   City
0  Alice  25  New York
1   Bob   30  Los Angeles
2  Charlie 35   Chicago
3   David 40   Houston
\nDataFrame 信息:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Name    4 non-null      object
1   Age      4 non-null      int64
2   City     4 non-null      object
dtypes: int64(1), object(2)
memory usage: 228.0+ bytes
\n选择 'Name' 列:
0    Alice
1     Bob
2  Charlie
3    David
Name: Name, dtype: object
\n年龄大于 30 的人:
   Name  Age   City
2  Charlie 35  Chicago
3   David 40  Houston

```

4.3.3. Matplotlib: 绘图和可视化

Matplotlib 是一个用于在 Python 中创建静态、动画和交互式可视化的综合库。它具有高度可定制性，并提供了种类繁多的绘图和图表。



```

# 按照惯例, pyplot 模块被导入并简写为 'plt'
import matplotlib.pyplot as plt
import numpy as np

# 中文显示支持
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

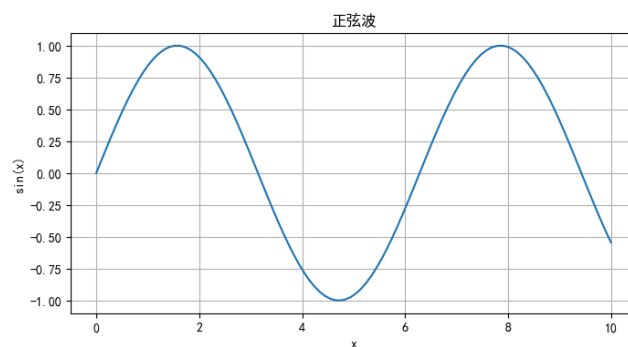
# 简单的线图
x_values = np.linspace(0, 10, 100) # 从 0 到 10 的 100 个点
y_values = np.sin(x_values)

plt.figure(figsize=(8, 4)) # 创建一个特定大小的图形
plt.plot(x_values, y_values)
plt.title('正弦波')
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.grid(True)
plt.show() # 显示绘图

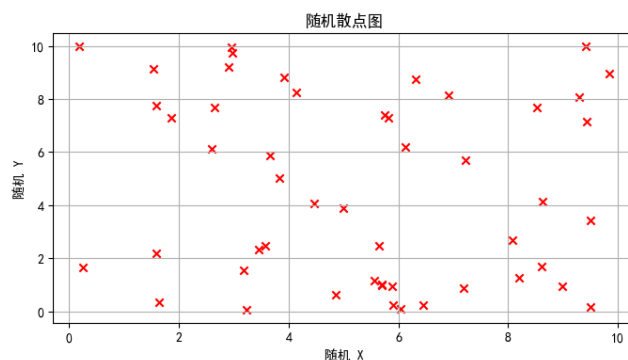
# 散点图
x_random = np.random.rand(50) * 10
y_random = np.random.rand(50) * 10

plt.figure(figsize=(8, 4))
plt.scatter(x_random, y_random, c='red', marker='x')
plt.title('随机散点图')
plt.xlabel('随机 X')
plt.ylabel('随机 Y')
plt.grid(True)
plt.show()

```



../_images/f11a91539af306f72e83d9bf661828a05df4e589a6641d2f803cae78fa245903.png



4.3.4. SciPy: 科学计算

SciPy 建立在 NumPy 之上，为数值积分、插值、优化、线性代数和统计等任务提供了大量用户友好且高效的数值例程。

让我们看一个简单的例子：使用 `scipy.optimize` 寻找函数的最小值。

```
from scipy.optimize import minimize
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 定义一个我们想要最小化的函数
# 让我们使用一个简单的二次函数：f(x) = (x - 3)^2
def objective_function(x):
    return (x - 3)**2

# 我们需要一个最小值的初始猜测值
initial_guess = 0.0

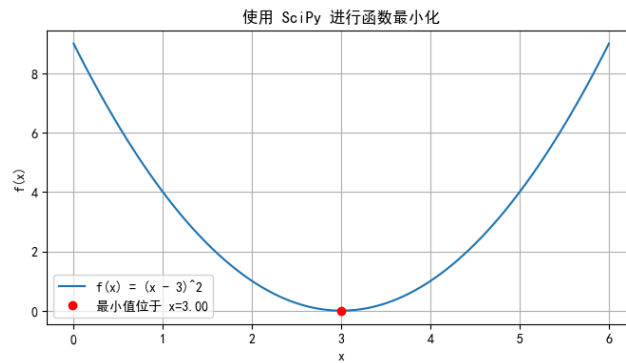
# 调用 minimize 函数
result = minimize(objective_function, initial_guess)

# result 对象包含有关优化的信息
if result.success:
    found_minimum = result.x[0]
    print(f"函数的最小值位于 x = {found_minimum:.4f}")
else:
    print("优化失败。")

# 我们可以绘制这个来验证
x_vals = np.linspace(0, 6, 100)
y_vals = objective_function(x_vals)

plt.figure(figsize=(8, 4))
plt.plot(x_vals, y_vals, label='f(x) = (x - 3)^2')
plt.plot(found_minimum, objective_function(found_minimum), 'ro', label=f'最小值位于')
plt.title('使用 SciPy 进行函数最小化')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.grid(True)
plt.show()
```

函数的最小值位于 $x = 3.0000$



../_images/3ad7d3c3efc5f78dd6e32556b1db847552147f3f5a8d48c15acf2fbd87b181c2.png

4.3.5. CoolProp: 热物理性质

CoolProp 是一个重要的开源库，为工程师和科学家提供各种流体和混合物的精确热物理性质数据。它广泛应用于热力学、传热和流体动力学模拟。

使用 CoolProp，您可以轻松计算水、制冷剂和工业气体等物质的密度、粘度、焓、熵等性质。

```
# 现在让我们使用 CoolProp 获取水的一些性质。
import CoolProp.CoolProp as CP

# 定义流体的状态
# 我们将使用大气压 (101325 帕斯卡) 和 100°C (373.15 开尔文) 的水
pressure = 101325 # Pa
temperature = 373.15 # K
fluid = 'Water'

# 获取此状态下饱和液态水的密度
density_l = CP.PropsSI('D', 'T', temperature, 'P', pressure, fluid)
print(f"液态水的密度: {density_l:.2f} kg/m^3")

# 获取此状态下饱和蒸汽的密度
density_v = CP.PropsSI('D', 'T', temperature, 'Q', 1, fluid) # Q=1 表示饱和蒸汽
print(f"蒸汽的密度: {density_v:.4f} kg/m^3")

# 获取饱和液态水的比焓 (能量)
enthalpy_l = CP.PropsSI('H', 'T', temperature, 'P', pressure, fluid)
print(f"液态水的焓: {enthalpy_l/1000:.2f} kJ/kg")

# 获取饱和蒸汽的比焓
enthalpy_v = CP.PropsSI('H', 'T', temperature, 'Q', 1, fluid)
print(f"蒸汽的焓: {enthalpy_v/1000:.2f} kJ/kg")

# 计算汽化潜热
latent_heat = enthalpy_v - enthalpy_l
print(f"汽化潜热: {latent_heat/1000:.2f} kJ/kg")
```

液态水的密度: 0.60 kg/m³
蒸汽的密度: 0.5982 kg/m³
液态水的焓: 2675.58 kJ/kg
蒸汽的焓: 2675.57 kJ/kg
汽化潜热: -0.01 kJ/kg

4.4. 建议

建议完整阅读本部分讲义的内容，并动手实践每个代码示例。通过实际操作，您将更好地理解 Python 语言及其科学计算库的强大功能。